TECH TALK · 2026

Scroll to Summary

The making of a cross-browser AI extension

Bharat Nanda

Principal Engineer

About Me

Bharat Nanda

Principal Engineer

Experience

18+ years of industry experience

Enterprise architect

Enterprise cloud architect building and designing large scale systems

Builder at heart

Ship things to scratch itches.
Problem first, polish second.

Agenda

01 The Problem & the spark

02 The idea in action

03 Architecture Deep Dive

04 Cross-Browser Challenges

05 Prompt Engineering & Evals

06 Security by Design

07 Lessons Learned

08 Live Demo + Q&A

The problem & the spark

01

It started with a 20-minute Medium article at 8PM

The problem

Too many steps for something simple

> **It was 8 PM.**
> A 20-minute Medium article about **MCP**
> was open on my laptop…
> **but I was too tired to start.**

The 'just use ChatGPT' approach:

Copy URL → **Open ChatGPT or Gemini** → **Write a prompt to summarize the page** → **Wait…** → **Read the summary**

Too much friction for something so simple.

The Idea

What if the browser plugin did the work?

One click. Any page. Instant AI summary.

No copy-paste

Extension reads the page content automatically

No prompt writing

Built-in smart prompt, tuned for web content

Choice of AI provider

OpenAI, Gemini, Azure, or even local Ollama

Saves summaries

Navigate back to any past summary anytime

Cross-browser

Chrome, Firefox, Safari, Edge — everyone wins

Privacy-first

Nothing leaves your machine except the page text to your chosen AI

The idea in action

A quick look at the plugin before diving in

02

Page Summarizer — feature overview

v1.1.0 · Published on Chrome and Mozilla Store

One-Click Summary

- Click the extension icon
- Hit 'Summarize'
- Summary opens in a new tab

Multi-Provider

- OpenAI (GPT-4o-mini)
- Google Gemini 2.5 Flash
- Azure OpenAI
- Ollama (100% local)

History & Cache

- All summaries auto-saved
- 30-min cache to save API costs
- Browse, revisit, delete

Security

- 40+ blacklisted domains
- Banking, email, gov sites
- Configurable by user

Smart Extraction

- Removes ads, navbars, footers
- Shadow DOM support
- JSON-LD fallback

Cross-Browser

- Chrome / Edge
- Firefox
- Safari (MV3 build)

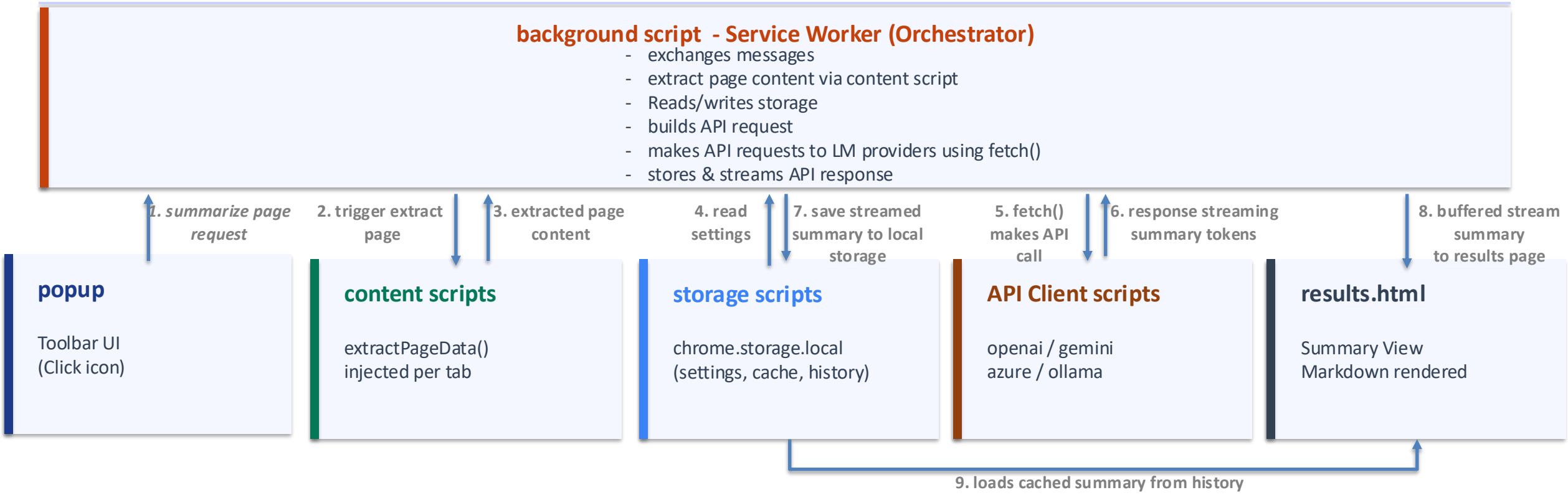
Architecture Deep Dive

03

How all the pieces fit together

Extension architecture

MV3 · Content Script · Service Worker · Storage



platform.js (entry point) — cross-browser shim that normalizes chrome. vs browser.* APIs (storage, tabs, scripting, alarms, notifications)*

Optimized summarization flow

What happens when you click 'Summarize'

Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
Blacklist Check	**Cache Lookup**	**Page Content Extract**	**Prompt Build**	**API Call (streaming)**	**Save & Display**
Is this domain on the blocked list? If yes → abort with friendly message.	Is there a fresh summary (< 30 min) for this exact URL? If yes → show it.	Inject content script → extractPageData() Remove junk, score DOM, clamp to token limit.	Wrap content in the summarisation prompt. Adapt length target based on page type.	Send to selected provider. Stream tokens back as SSE or NDJSON.	Auto-save to history. Cache result. Open results.html in new tab.

All orchestrated by background.js and summarySession class — encapsulates the entire flow, keeping popup.js clean and each step independently testable.

**Cache key = provider + model + language + URL → no stale cross-provider summaries*

Cross-Browser Challenges

04

Where 80% of my pain lived

Manifest V3 — the new reality

Chrome pushed MV3; the ecosystem scrambled

What MV3 changes:

- Service Worker replaces persistent background page
- No DOM access in service worker
- Strict CSP — no remote code execution
- declarativeNetRequest (not webRequest) for blocking
- 5-minute idle termination of service worker

How we handled it:

✓ Content script injection

Use chrome.scripting.executeScript to inject extractPageData() into each tab on demand

✓ platform.js shim

Wraps chrome.* and browser.* into a unified API so core code stays browser-agnostic

✓ Module service worker

background.js uses 'type: module' to keep the code organized and importable

✓ Alarm API for cleanup

chrome.alarms replaces setInterval for periodic cache expiry tasks

Firefox & Safari — challenges

Each browser has its own personality

Chrome / Edge

- ✓ service_worker background type
- ✓ Side panels supported
- ✓ Full MV3 support
- ✓ Easiest dev experience
- ⚠ host_permissions required

Firefox

- ✗ background.scripts (not service_worker)
- ✗ No side panel API
- ✓ browser.* namespace
- ⚠ gecko section in manifest required
- ⚠ websiteContent permission needed

Safari

- ✗ streaming not efficient
- ✗ No side panel
- ✓ Xcode build step required
- ⚠ Different local storage limits
- ⚠ Response buffering issues

Solution: separate /platforms/ manifest files + platform.js shim → one shared /core/ codebase, three browser builds

Content extraction — the hard part

The web is messy. Pages are noisy. Signal is rare.

Smart Content Extraction

contentExtractor.js

Extraction Waterfall (tries in order):

1. JSON-LD articleBody

Some pages embed clean structured data.
Fastest, highest quality.

2. pickBestDomRoot()

Score every <article>, <main>,
by content density, position, link ratio.

3. Shadow DOM scan

Some SPAs hide content in web components.
Walk shadow roots up to depth 18.

4. Raw body fallback

Last resort: document.body.innerText
after junk selectors removed.

```
score = (
  textLength
  + paragraphs    × 450
  + headings      × 260
  + list items    × 110
  + centerBoost   (horizontal)
  + topBoost      (vertical pos)
  + widthBoost
  + articleTag?   +1200
)
- linkDensity penalty
- formElements   × 160
- multiArticle   × 2000

# Junk selector removes:
nav, footer, header, aside
.ad, .promo, .sponsored
.modal, .popup, .paywall
```

Token Optimisation

Because every token costs money (and time)

Problem: Raw page text can be hundreds of thousands of characters

Junk Removal

Strip navbars, footers, ads, cookie banners before text even leaves the DOM

Content Scoring

Only extract the highest-signal DOM subtree — not the whole page body

Per-Provider Clamp

Gemini/Ollama → 30k chars
OpenAI/Azure → 50k chars

Deduplication

seen Set prevents the same paragraph being sent twice

Selection Priority

If user selects text → summarize that only — precision over volume

Compact Profile

Optional compact prompt: 180 words max — for quick reads or cost-sensitive users

Result: Consistent, fast, cost-efficient summaries — even on Wikipedia and dense documentation pages

Prompt Engineering & Evals

05

How do you know your prompt is good?

The prompt engineering journey

Many drafts. Many failures. One winner.

v0.1	v0.2	v0.3	v0.4	v1.0	v1.1
Just summarize this page	**Summarize in bullet points**	**+ page type detection**	**+ adaptive word count**	**Full constraint set**	**+ compact profile option**
No structure. Wall of text. Useless.	Better. But too long on short pages, too short on complex docs.	Article vs product vs docs. Much more relevant output.	80–250 words based on page complexity. Context-aware length.	Hard limits, output rules, junk exclusions, bold usage guidelines.	Separate 180-word bullet-only mode for power users.

The winning prompt structure (v1.0+):

Role: 'You are a professional summarizer'
Hard constraints (word count, formatting rules)
Length target (adaptive: 80–250 words based on page type)
Page-type handlers (article / docs / product / homepage)
What to include / what to strictly exclude
Output rules (no preamble, no follow-up questions, no headings)

What Are Evals?

Testing your prompts like you test your code

Evals (Evaluations) = a structured way to measure whether your LLM prompt is producing good outputs — consistently, across many inputs.

Think of evals like unit tests — but for prompt quality, not code correctness.

Without Evals

- You change the prompt
- It 'seems' better on 2 examples
- You ship it
- It regresses on edge cases
- You don't know why

With Evals

- Maintain eval_set.csv of test pages
- Run prompt against all examples
- Score each output (auto or human)
- Compare versions objectively
- Ship with confidence

What to measure

- Word count (within target?)
- No preamble / no closing remarks?
- Key facts captured?
- No hallucinations?
- Language match?

Running evals in practice

How I built and used eval_set.csv

Step 1 — Build a diverse dataset:

```
Page Type           | URL                      | Expected Behavior
————————————————————|——————————————————————————|————————————————————————————
Tech article        | medium.com/...           | Bullet points, 150-200 words
Product landing     | some-saas.com            | What it is, features, CTA
Documentation       | docs.python.org          | Purpose, key concepts, usage
Short blog post     | dev.to/...               | 80-130 words, core thesis
Homepage / feed     | news.ycombinator.com     | Digest of top items, no narrative
Dense academic      | arxiv.org/...            | 200-250 words, key findings
```

Step 2 — Score each output:

```
Word count in expected range?          → pass/fail
Output starts with bullet '-' ?        → pass/fail
No preamble ('Here is a summary...')?  → pass/fail
Key entity from page mentioned?        → human judgement (0-3)
Hallucination check?                   → human judgement (0-3)
```

Security by Design

Extension security is non-obvious — here's what I learned

06

Security Layers

Defence in depth for a browser extension

Sensitive page summarisation	Risk: User accidentally sends banking / email content to external AI	Fix: Domain blacklist — 40+ default entries covering banks, mail, gov, military. Wildcard & regex support.
Prompt injection via page content	Risk: Malicious page embeds 'Ignore all instructions...'	Fix: Content extractor strips <script>, iframes, hidden elements. Only semantic HTML text reaches the prompt.
API key leakage	Risk: Keys stored insecurely or sent to intermediate server	Fix: chrome.storage.local only. Direct fetch() to provider endpoints. No server, no proxy, no middleman.
XSS in summary rendering	Risk: LLM output contains HTML/JS injected into results page	Fix: Markdown renderer sanitizes output. No innerHTML with raw LLM text. Template-based rendering.

Privacy policy published. No telemetry. No analytics. Everything runs in your browser.

Domain Blacklist — under the hood

Compiled regexes, wildcard expansion, exact sets

Pattern types supported:

Exact match	`bankofamerica.com`	Set<string> lookup — O(1)
Wildcard apex	`*.hsbc.com`	Suffix check: hostname.endsWith('.hsbc.com')
Complex wildcard	`*.gov.*`	Compiled to /^.*\.gov\..*$/i regex
Custom user list	`*.mycompany.intranet`	Merged with default list at runtime

Performance:

Blacklist is compiled once and cached in a Map keyed by the combined list string.
Subsequent checks hit the cache — no re-parsing on every URL.

Default coverage includes:

- Major banks (global)
- Email services
- Government & military
- Payroll & tax
- Productivity suites
- Cloud storage

Lessons Learned

What I'd do the same. What I'd do differently.

07

Lessons Learned

Hard-won truths from 6 months of side-project hacking

Start with one browser, then expand

What I'd change

I started with Chrome-only then retrofitted Firefox & Safari.
Starting with the abstraction (platform.js) from day 1 would have saved weeks.

Evals should come before prompt v2

What I'd change

I iterated on prompts by feel for too long. Setting up an eval set earlier would have made improvements objective, not subjective.

Content extraction is the secret sauce

Validated

Users don't care about the LLM provider. They care if the summary is accurate. That accuracy lives or dies in extraction quality.

MV3 service workers are tricky — embrace alarms

Validated

Background service workers can die mid-task. Alarms API is your friend for anything periodic. Don't fight the platform.

Security defaults matter more than configurability

Validated

Ship with safe defaults (blacklist, local storage, no telemetry).
Users trust extensions they don't have to think about.

Ship early, iterate in public

Validated

The Chrome Web Store submission forced me to clean up the codebase.
The threat of real users makes you write better error messages.

What's Next

The roadmap — v2 ideas

Anthropic Claude support

Add Claude to the provider list.
API is clean — should be a quick add.

Automated eval pipeline

GitHub Action that runs eval_set.csv
on every prompt change. CI for prompts.

Side panel (Chrome)

Persistent panel instead of popup.
Better UX for heavy readers.

Chat & ask

ask a question about the article.
Not just 'summarize' — but 'explain' or
'critique'.

Webhook / export

Send summaries to Notion, Obsidian, or
a webhook for PKM integrations.

Live Demo

Let's see it in action

08

Key Takeaways

What I hope you take away from this talk

Scratch your own itch

The best side projects come from genuine frustration.
Users like you → better product intuition.

Browser APIs have teeth

MV3 + cross-browser compatibility is a real challenge.
Abstract early. Platform.js pattern is your friend.

Extraction beats prompting

Garbage in, garbage out. Fix the input first.
A great prompt on bad content is still bad.

Evals are not optional

Once you care about quality, you need a way to
measure it. Eval set = test suite for your prompts.

Security as defaults

Don't make users think about security.
Ship with the blacklist on. Privacy by design.

Ship and learn

The Chrome store submission was scary.
It was also the best forcing function I've ever had.

Thank You

GitHub: https://github.com/bharatnanda/page-summariser

Chrome Store: https://chromewebstore.google.com/detail/page-summarizer/ehnjnlmphpcoijjahimaggoccnjdhaml

Mozilla Store: https://addons.mozilla.org/en-US/firefox/addon/page-summarizer-extension/